



KATY CYBERWISE CLUB · PROJECT 1

AI-Powered Network Intrusion Detection System

Captures network flows, scores them with two machine-learning models (Isolation Forest + Random Forest), streams alerts to a live dashboard, and evaluates detection quality.

Complete source code · Generated July 26, 2026

Contents — 16 files

1. `app.py`
2. `evaluate.py`
3. `test_ids.py`
4. `train.py`
5. `train_supervised.py`
6. `ids/__init__.py`
7. `ids/alerting.py`
8. `ids/capture.py`
9. `ids/datasets.py`
10. `ids/features.py`
11. `ids/generator.py`
12. `ids/model.py`
13. `ids/supervised.py`
14. `templates/dashboard.html`
15. `metrics.txt`
16. `requirements.txt`

```
1 """
2 Project 1 – AI-Powered Network Intrusion Detection System
3 =====
4
5 A REAL, runnable Flask application:
6
7 * loads a trained IsolationForest model (train.py)
8 * scores network flows for anomalies in real time
9 * streams alerts to a live dashboard via Server-Sent Events (SSE)
10 * accepts real flows from the Scapy capture bridge at POST /api/flow
11 * falls back to a built-in synthetic generator so the demo runs anywhere
12
13 Run:
14     pip install -r requirements.txt
15     python train.py           # builds model.pkl (once)
16     python app.py             # open http://127.0.0.1:5001
17
18 Feed it REAL traffic from your lab (needs Scapy):
19     python -m ids.capture --pcap lab.pcap --url http://127.0.0.1:5001
20     sudo python -m ids.capture --iface eth0 --url http://127.0.0.1:5001
21 """
22 import json
23 import os
24 import queue
25 import random
26 import sqlite3
27 import threading
28 import time
29
30 from flask import Flask, Response, render_template, request, jsonify, g
31
32 from ids.model import IDSMModel, MODEL_PATH
33 from ids import generator
34
35 app = Flask(__name__)
36 DB_PATH = os.path.join(os.path.dirname(__file__), "alerts.db")
37
38 # ---- load or lazily train the model -----
39 if not os.path.exists(MODEL_PATH):
40     print("model.pkl not found – training a quick model now ...")
41     import train
42     train.main()
43 MODEL = IDSMModel.load()
44
45 # ---- optional supervised model (Phase 4): names the attack type live -----
46 RF = None
47 try:
48     from ids.supervised import RFModel, MODEL_PATH as RF_PATH
49     if os.path.exists(RF_PATH):
50         RF = RFModel.load()
51         print("Loaded Random Forest classifier (rf_model.pkl) for attack typing.")
52 except Exception as _e:
53     RF = None
54
55 # ---- real-time alerting (Phase 6) -----
56 from ids import alerting
57
58 # ---- simple in-memory pub/sub so many dashboard tabs can subscribe -----
59 _subscribers = []
60 _sub_lock = threading.Lock()
61
62 # ---- demo generator state -----
63 STATE = {"running": True, "interval": 0.7, "inject": 0}
64 _rng = random.Random()
65 _counter = {"flows": 0, "alerts": 0}
66
67
68 def _db():
69     conn = sqlite3.connect(DB_PATH)
70     conn.row_factory = sqlite3.Row
71     return conn
72
73
74 def init_db():
75     conn = _db()
76     conn.execute("""CREATE TABLE IF NOT EXISTS alerts(
77         id INTEGER PRIMARY KEY AUTOINCREMENT, ts REAL, src TEXT, dst TEXT,
78         dst_port INTEGER, proto TEXT, bytes INTEGER, packets INTEGER,
79         suspicion REAL, label TEXT)""")
80     conn.commit()
81     conn.close()
82
83
84 def publish(event: dict):
85     with _sub_lock:
86         for q in list(_subscribers):
87             try:
88                 q.put_nowait(event)
89             except queue.Full:
90                 pass
91
92
93 def process_flow(flow: dict, source: str):
94     """Score a flow, count it, persist alerts, and publish to dashboards."""
95     suspicion, is_anom = MODEL.score(flow) # Isolation Forest (anomaly)
96     # If the supervised model is available, let IT name the attack type – this
97     # is the model's own verdict, not the ground-truth label.
98     if is_anom and RF is not None:
99         try:
100             verdict = RF.score_flow(flow)
101         except Exception:
102             verdict = flow.get("label", "attack")
103     else:
104         verdict = flow.get("label", "attack" if is_anom else "normal")
105     _counter["flows"] += 1
106     evt = {
107         "ts": time.time(), "src": flow.get("src", "?"), "dst": flow.get("dst", "?"),
108         "dst_port": int(flow.get("dst_port", 0)), "proto": flow.get("proto", "TCP"),
109         "bytes": int(flow.get("bytes", 0)), "packets": int(flow.get("packets", 0)),
110         "suspicion": round(suspicion, 3), "anomaly": is_anom,
111         "label": verdict,
112         "source": source, "total_flows": _counter["flows"],
113     }
114     if is_anom:
115         _counter["alerts"] += 1
116         conn = _db()
117         conn.execute("""INSERT INTO alerts(ts,src,dst,dst_port,proto,bytes,packets,suspicion,label)
118             VALUES(?, ?, ?, ?, ?, ?, ?, ?)""",
119             (evt["ts"], evt["src"], evt["dst"], evt["dst_port"], evt["proto"],
120              evt["bytes"], evt["packets"], evt["suspicion"], evt["label"]))
121         conn.commit()
122         conn.close()
123     evt["total_alerts"] = _counter["alerts"]
124     if is_anom:
125         alerting.maybe_alert(evt) # desktop / email / log (off unless IDS_ALERTS=1)
126     publish(evt)
127     return evt
128
129
130 # ---- background demo generator -----
131 def demo_loop():
132     while True:
133         if STATE["running"]:
134             if STATE["inject"] > 0:
135                 flow = generator.attack_flow(rng=_rng)
136                 STATE["inject"] -= 1
137             else:
```

```

138         flow = (generator.attack_flow(rng=_rng)
139                 if _rng.random() < 0.08 else generator.normal_flow(_rng))
140         process_flow(flow, source="demo")
141         time.sleep(STATE["interval"])
142
143
144 # ---- routes -----
145 @app.route("/"):
146     def dashboard():
147         return render_template("dashboard.html")
148
149
150 @app.route("/favicon.ico")
151 def favicon():
152     return ("", 204)
153
154
155 @app.route("/stream")
156 def stream():
157     def gen():
158         q = queue.Queue(maxsize=100)
159         with _sub_lock:
160             _subscribers.append(q)
161         try:
162             yield "retry: 3000\n\n"
163             while True:
164                 try:
165                     evt = q.get(timeout=15)
166                     yield f"data: {json.dumps(evt)}\n\n"
167                 except queue.Empty:
168                     yield ": keep-alive\n\n"
169             finally:
170                 with _sub_lock:
171                     if q in _subscribers:
172                         _subscribers.remove(q)
173         return Response(gen(), mimetype="text/event-stream",
174                         headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"})
175
176
177 @app.route("/api/flow", methods=["POST"])
178 def api_flow():
179     """Ingest a REAL flow from the Scapy capture bridge (or any producer)."""
180     flow = request.get_json(force=True, silent=True) or {}
181     evt = process_flow(flow, source="capture")
182     return jsonify({"ok": True, "suspicion": evt["suspicion"], "anomaly": evt["anomaly"]})
183
184
185 @app.route("/control", methods=["POST"])
186 def control():
187     action = (request.form.get("action") or request.json.get("action"))
188     if request.is_json else request.form.get("action")
189     if action == "pause":
190         STATE["running"] = False
191     elif action == "resume":
192         STATE["running"] = True
193     elif action == "attack":
194         STATE["inject"] = 6
195     elif action == "speed":
196         STATE["interval"] = float(request.values.get("value", 0.7))
197     return jsonify({"ok": True, "state": {k: STATE[k] for k in ("running", "interval")}})
198
199
200 @app.route("/alerts")
201 def alerts():
202     conn = _db()
203     rows = conn.execute("SELECT * FROM alerts ORDER BY id DESC LIMIT 100").fetchall()
204     conn.close()
205     return jsonify([dict(r) for r in rows])
206
207
208 @app.route("/stats")
209 def stats():
210     return jsonify({"flows": _counter["flows"], "alerts": _counter["alerts"],
211                   "threshold": MODEL.threshold})
212
213
214 if __name__ == "__main__":
215     init_db()
216     threading.Thread(target=demo_loop, daemon=True).start()
217     port = int(os.environ.get("PORT", "5001"))
218     print(f"\n IDS dashboard at http://127.0.0.1:{port} (model threshold={MODEL.threshold:.3f})\n")
219     app.run(host="127.0.0.1", port=port, threaded=True, debug=False)

```

```
1  """Phase 5 - rigorous evaluation + publication-style figures.
2
3  Trains both models on the same data and produces, in figures/:
4  1. roc_curves.png          ROC curve: Random Forest vs Isolation Forest
5  2. pr_curves.png           Precision-Recall curve for both
6  3. confusion_matrix.png     Random Forest, per attack type
7  4. feature_importance.png  what the model relies on
8  5. per_attack_recall.png    detection rate by attack type, both models
9  6. threshold_tuning.png     Isolation Forest: precision/recall/F1 vs threshold
10 7. model_comparison.png     ROC-AUC & average-precision, side by side
11 Also writes metrics.txt with the headline numbers and a short interpretation.
12
13     python evaluate.py
14
15 Swap in real data by editing build_flows() (or load NSL-KDD / lab captures).
16 """
17 import os
18 import random
19
20 import numpy as np
21 import matplotlib
22 matplotlib.use("Agg")
23 import matplotlib.pyplot as plt
24 from sklearn.model_selection import train_test_split
25 from sklearn.metrics import (roc_curve, auc, precision_recall_curve,
26                             average_precision_score, confusion_matrix,
27                             precision_recall_fscore_support)
28
29 from ids import generator
30 from ids.features import FEATURES, flow_to_vector
31 from ids.model import IDModel
32 from ids.supervised import RFModel
33
34 FIG = os.path.join(os.path.dirname(__file__), "figures")
35 os.makedirs(FIG, exist_ok=True)
36
37 # CyberWise brand palette
38 TEAL, ROSE, GOLD, SEA, VIOLET, INK = "#2f8fb0", "#e2555f", "#d9a441", "#3f9e83", "#8a6fc0", "#22333f"
39 plt.rcParams.update({
40     "figure.facecolor": "white", "axes.facecolor": "white",
41     "axes.edgecolor": "#cbd5db", "axes.labelcolor": INK, "text.color": INK,
42     "xtick.color": INK, "ytick.color": INK, "axes.grid": True,
43     "grid.color": "#e6ecef", "font.size": 11, "axes.titleweight": "bold",
44 })
45
46
47 def build_flows(n_normal=3000, n_attack_each=600, seed=7):
48     rng = random.Random(seed)
49     flows, labels = [], []
50     for _ in range(n_normal):
51         flows.append(generator.normal_flow(rng)); labels.append("normal")
52     for kind in generator.ATTACK_TYPES:
53         for _ in range(n_attack_each):
54             flows.append(generator.attack_flow(kind, rng)); labels.append(kind)
55     return flows, labels
56
57
58 def main():
59     flows, labels = build_flows()
60     idx = list(range(len(flows)))
61     tr, te = train_test_split(idx, test_size=0.3, random_state=1, stratify=labels)
62
63     tr_flows = [flows[i] for i in tr]; tr_labels = [labels[i] for i in tr]
64     te_flows = [flows[i] for i in te]; te_labels = [labels[i] for i in te]
65
66     # --- train both models on the SAME training data ---
67     # Isolation Forest: unsupervised, trains on normal only
68     if_model = IDModel.train([f for f, l in zip(tr_flows, tr_labels) if l == "normal"])
69     # Random Forest: supervised, trains on everything
70     rf_model = RFModel.train([flow_to_vector(f) for f in tr_flows], tr_labels)
71
72     # --- score the shared test set ---
73     y_true = np.array([0 if l == "normal" else 1 for l in te_labels])
74     if_score = np.array([if_model.score(f)[0] for f in te_flows]) # suspicion 0..1
75     Xte = [flow_to_vector(f) for f in te_flows]
76     rf_score = rf_model.attack_probability(Xte) # P(attack)
77     rf_pred = rf_model.predict(Xte)
78
79     # ----- 1. ROC -----
80     plt.figure(figsize=(6.2, 5))
81     for name, score, col in [("Random Forest", rf_score, TEAL), ("Isolation Forest", if_score, ROSE)]:
82         fpr, tpr, _ = roc_curve(y_true, score); a = auc(fpr, tpr)
83         plt.plot(fpr, tpr, color=col, lw=2.4, label=f"{name} (AUC = {a:.3f})")
84     plt.plot([0, 1], [0, 1], "--", color="#9fb0b8", lw=1)
85     plt.xlabel("False positive rate"); plt.ylabel("True positive rate (attacks caught)")
86     plt.title("ROC - attack detection"); plt.legend(loc="lower right")
87     plt.tight_layout(); plt.savefig(f"{FIG}/roc_curves.png", dpi=150); plt.close()
88
89     # ----- 2. Precision-Recall -----
90     plt.figure(figsize=(6.2, 5))
91     for name, score, col in [("Random Forest", rf_score, TEAL), ("Isolation Forest", if_score, ROSE)]:
92         pr, rc, _ = precision_recall_curve(y_true, score)
93         ap = average_precision_score(y_true, score)
94         plt.plot(rc, pr, color=col, lw=2.4, label=f"{name} (AP = {ap:.3f})")
95     plt.xlabel("Recall"); plt.ylabel("Precision")
96     plt.title("Precision-Recall - attack detection"); plt.legend(loc="lower left")
97     plt.tight_layout(); plt.savefig(f"{FIG}/pr_curves.png", dpi=150); plt.close()
98
99     # ----- 3. Confusion matrix (RF, multiclass) -----
100     classes = sorted(set(labels))
101     cm = confusion_matrix(te_labels, rf_pred, labels=classes)
102     plt.figure(figsize=(6.8, 5.6))
103     im = plt.imshow(cm, cmap="GnBu")
104     plt.colorbar(im, fraction=0.046, pad=0.04)
105     plt.xticks(range(len(classes)), classes, rotation=45, ha="right")
106     plt.yticks(range(len(classes)), classes)
107     thresh = cm.max() * 0.6
108     for i in range(len(classes)):
109         for j in range(len(classes)):
110             plt.text(j, i, cm[i, j], ha="center", va="center",
111                     color="white" if cm[i, j] > thresh else INK, fontsize=9)
112     plt.ylabel("True label"); plt.xlabel("Predicted label")
113     plt.title("Random Forest - confusion matrix")
114     plt.tight_layout(); plt.savefig(f"{FIG}/confusion_matrix.png", dpi=150); plt.close()
115
116     # ----- 4. Feature importance -----
117     imp = rf_model.feature_importances()
118     feats = sorted(imp, key=imp.get)
119     plt.figure(figsize=(6.4, 4.4))
120     plt.bar(feats, [imp[f]*100 for f in feats], color=TEAL)
121     plt.xlabel("Importance (%)"); plt.title("What the model relies on")
122     for i, f in enumerate(feats):
123         plt.text(imp[f]*100 + 0.5, i, f"{imp[f]*100:.1f}%", va="center", fontsize=9)
124     plt.tight_layout(); plt.savefig(f"{FIG}/feature_importance.png", dpi=150); plt.close()
125
126     # ----- 5. Per-attack-type recall (both models) -----
127     attack_types = generator.ATTACK_TYPES
128     if_rec, rf_rec = [], []
129     for kind in attack_types:
130         mask = np.array([l == kind for l in te_labels])
131         if mask.sum() == 0:
132             if_rec.append(0); rf_rec.append(0); continue
133         if_rec.append(np.mean(if_score[mask] >= if_model.threshold) * 100)
134         rf_rec.append(np.mean(rf_score[mask] >= 0.5) * 100)
135     x = np.arange(len(attack_types)); w = 0.38
136     plt.figure(figsize=(7.6, 4.6))
137     plt.bar(x - w/2, rf_rec, w, label="Random Forest", color=TEAL)
```

```

138 plt.bar(x + w/2, if_rec, w, label="Isolation Forest", color=R0SE)
139 plt.xticks(x, attack_types, rotation=20, ha="right")
140 plt.ylabel("Detection rate (%)"); plt.ylim(0, 105)
141 plt.title("Detection rate by attack type"); plt.legend()
142 plt.tight_layout(); plt.savefig(f"{FIG}/per_attack_recall.png", dpi=150); plt.close()
143
144 # ----- 6. Threshold tuning (Isolation Forest) -----
145 ths = np.linspace(0.05, 0.95, 37)
146 P, R, F = [], [], []
147 for t in ths:
148     pred = (if_score >= t).astype(int)
149     p, r, f, _ = precision_recall_fscore_support(y_true, pred, average="binary", zero_division=0)
150     P.append(p); R.append(r); F.append(f)
151 plt.figure(figsize=(6.6, 4.6))
152 plt.plot(ths, P, color=TEAL, lw=2, label="Precision")
153 plt.plot(ths, R, color=R0SE, lw=2, label="Recall")
154 plt.plot(ths, F, color=GOLD, lw=2.4, label="F1")
155 plt.axvline(if_model.threshold, ls="--", color="#9fb0b8", label=f"chosen = {if_model.threshold:.2f}")
156 plt.xlabel("Suspicion threshold"); plt.ylabel("Score")
157 plt.title("Isolation Forest – tuning the alert threshold"); plt.legend()
158 plt.tight_layout(); plt.savefig(f"{FIG}/threshold_tuning.png", dpi=150); plt.close()
159
160 # ----- 7. Model comparison -----
161 rf_auc = auc(*roc_curve(y_true, rf_score)[:2]); if_auc = auc(*roc_curve(y_true, if_score)[:2])
162 rf_ap = average_precision_score(y_true, rf_score); if_ap = average_precision_score(y_true, if_score)
163 plt.figure(figsize=(6.2, 4.4))
164 metrics = ["ROC-AUC", "Avg precision"]; xm = np.arange(len(metrics)); w = 0.36
165 plt.bar(xm + w/2, [rf_auc, rf_ap], w, label="Random Forest", color=TEAL)
166 plt.bar(xm + w/2, [if_auc, if_ap], w, label="Isolation Forest", color=R0SE)
167 for i, v in enumerate([rf_auc, rf_ap]): plt.text(i - w/2, v + .01, f"{v:.3f}", ha="center", fontsize=9)
168 for i, v in enumerate([if_auc, if_ap]): plt.text(i + w/2, v + .01, f"{v:.3f}", ha="center", fontsize=9)
169 plt.xticks(xm, metrics); plt.ylim(0, 1.08); plt.ylabel("Score")
170 plt.title("Model comparison"); plt.legend()
171 plt.tight_layout(); plt.savefig(f"{FIG}/model_comparison.png", dpi=150); plt.close()
172
173 # ----- metrics.txt -----
174 with open(os.path.join(os.path.dirname(__file__), "metrics.txt"), "w") as fh:
175     fh.write("IDS evaluation summary\n===== \n\n")
176     fh.write(f"Test flows: {len(test_flows)} (normal + {len(attack_types)} attack types)\n\n")
177     fh.write(f"Random Forest ROC-AUC={rf_auc:.3f} AvgPrecision={rf_ap:.3f}\n")
178     fh.write(f"Isolation Forest ROC-AUC={if_auc:.3f} AvgPrecision={if_ap:.3f}\n\n")
179     fh.write("Detection rate by attack type (%):\n")
180     for k, r_rf, r_if in zip(attack_types, rf_rec, if_rec):
181         fh.write(f" {k:18s} RandomForest={r_rf:5.1f} IsolationForest={r_if:5.1f}\n")
182     fh.write("\nFeature importances (Random Forest):\n")
183     for f in sorted(imp, key=imp.get, reverse=True):
184         fh.write(f" {f:16s} {imp[f]*100:5.1f}%\n")
185     fh.write("\nNote: figures use labelled synthetic traffic so the pipeline is\n")
186     fh.write("fully reproducible. Numbers on real NSL-KDD or lab-captured traffic\n")
187     fh.write("will be lower/noisier – that is expected and more realistic.\n")
188
189 print("Wrote 7 figures to figures/ and metrics.txt")
190 print(f" Random Forest ROC-AUC={rf_auc:.3f} AP={rf_ap:.3f}")
191 print(f" Isolation Forest ROC-AUC={if_auc:.3f} AP={if_ap:.3f}")
192
193
194 if __name__ == "__main__":
195     main()

```

test_ids.py

56 lines

```
1  """Tests for the IDS: feature extraction, model training, and detection quality.
2
3      python test_ids.py
4  """
5  import random
6
7  from ids.features import flow_to_vector, FEATURES
8  from ids.generator import normal_flow, attack_flow, ATTACK_TYPES
9  from ids.model import IDSModel
10
11
12  def test_features():
13      f = {"bytes": 1000, "packets": 10, "duration": 2.0, "conn_rate": 5, "dst_port": 443}
14      v = flow_to_vector(f)
15      assert len(v) == len(FEATURES), "vector length must match FEATURES"
16      assert v[3] == 100.0, "mean_pkt_size should be bytes/packets"
17      return True
18
19
20  def test_detection():
21      rng = random.Random(1)
22      model = IDSModel.train([normal_flow(rng) for _ in range(2000)])
23      normal = [normal_flow(rng) for _ in range(500)]
24      attacks = [attack_flow(rng.choice(ATTACK_TYPES), rng) for _ in range(500)]
25      fp = sum(1 for f in normal if model.score(f)[1]) / len(normal)
26      tp = sum(1 for f in attacks if model.score(f)[1]) / len(attacks)
27      print(f" attack recall={tp*100:.1f}% false-positive rate={fp*100:.1f}%")
28      assert tp > 0.60, "should detect a majority of attacks"
29      assert fp < 0.15, "false-positive rate should be low"
30      return True
31
32
33  def test_save_load(tmp="model_test.pkl"):
34      import os
35      rng = random.Random(2)
36      m = IDSModel.train([normal_flow(rng) for _ in range(500)])
37      m.save(tmp)
38      m2 = IDSModel.load(tmp)
39      f = attack_flow("Data exfiltration", rng)
40      assert m.score(f)[0] == m2.score(f)[0], "loaded model must score identically"
41      os.remove(tmp)
42      return True
43
44
45  if __name__ == "__main__":
46      print("=== IDS tests ===")
47      ok = True
48      for name, fn in [("features", test_features), ("detection", test_detection), ("save/load", test_save_load)]:
49          try:
50              fn(): print(f" [PASS] {name}")
51          except AssertionError as e:
52              ok = False; print(f" [FAIL] {name}: {e}")
53      print("=====")
54      print("ALL GOOD" if ok else "SOME CHECKS FAILED")
55      raise SystemExit(0 if ok else 1)
```

train.py

45 lines

```
1  """Train the IDS anomaly model on synthetic NORMAL traffic and save model.pkl.
2
3      python train.py
4
5  Prints a quick evaluation on held-out normal + attack flows so you can see
6  detection quality (recall on attacks, false-positive rate on normal).
7  """
8  import random
9
10 from ids.generator import normal_flow, attack_flow, ATTACK_TYPES
11 from ids.model import IDSMoDel
12
13
14 def main(seed=42):
15     rng = random.Random(seed)
16     # 1) train on normal-only traffic
17     train_normal = [normal_flow(rng) for _ in range(3000)]
18     model = IDSMoDel.train(train_normal)
19     model.save()
20     print(f"Trained IsolationForest on {len(train_normal)} normal flows -> model.pkl")
21     print(f"Suspicion threshold: {model.threshold:.3f}")
22
23     # 2) evaluate
24     test_normal = [normal_flow(rng) for _ in range(1000)]
25     test_attacks = [attack_flow(rng.choice(ATTACK_TYPES), rng) for _ in range(1000)]
26
27     fp = sum(1 for f in test_normal if model.score(f)[1])
28     tp = sum(1 for f in test_attacks if model.score(f)[1])
29     print("\n=== Evaluation ===")
30     print(f"  Attack recall (detected):    {tp/len(test_attacks)*100:5.1f}%  ({tp}/{len(test_attacks)})")
31     print(f"  Normal false-positive rate:    {fp/len(test_normal)*100:5.1f}%  ({fp}/{len(test_normal)})")
32
33     # per-attack-type recall
34     print("  Recall by attack type:")
35     for kind in ATTACK_TYPES:
36         s = [attack_flow(kind, rng) for _ in range(200)]
37         d = sum(1 for f in s if model.score(f)[1])
38         print(f"      {kind:18s} {d/len(s)*100:5.1f}%")
39     print("=====")
40     return model
41
42
43 if __name__ == "__main__":
44     main()
```


train_supervised.py

56 lines

```
1  """Phase 4 – train and evaluate the supervised Random Forest classifier.
2
3      python train_supervised.py          # trains on synthetic labelled data
4      python train_supervised.py data/KDDTrain+.txt  # trains on real NSL-KDD
5
6  Prints precision / recall / F1 (per class + overall) and a text confusion
7  matrix, then saves rf_model.pkl. For the publication-style charts, run
8  evaluate.py afterwards.
9  """
10 import sys
11
12 from sklearn.model_selection import train_test_split
13 from sklearn.metrics import classification_report, confusion_matrix
14
15 from ids import datasets
16 from ids.supervised import RFModel
17
18
19 def main(nslkdd_path=None):
20     if nslkdd_path:
21         print(f>Loading real NSL-KDD data from {nslkdd_path} ...")
22         X, y = datasets.load_nslkdd(nslkdd_path)
23     else:
24         print("Building synthetic labelled dataset (normal + 5 attack types) ...")
25         X, y = datasets.synthetic_labeled()
26         print(f>Total samples: {len(X)}")
27
28     Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.30,
29                                         random_state=42, stratify=y)
30
31     model = RFModel.train(Xtr, ytr)
32     model.save()
33     print("Trained Random Forest -> rf_model.pkl\n")
34
35     pred = model.predict(Xte)
36     print("=== Classification report (test set) ===")
37     print(classification_report(yte, pred, digits=3, zero_division=0))
38
39     labels = sorted(set(y))
40     cm = confusion_matrix(yte, pred, labels=labels)
41     print("=== Confusion matrix (rows = true, cols = predicted) ===")
42     head = "".join(f"{l[:9]:>10}" for l in labels)
43     print(f"{'':>18}{head}")
44     for i, l in enumerate(labels):
45         print(f"{l:>18}" + "".join(f"{cm[i][j]:>10}" for j in range(len(labels))))
46
47     print("\n=== Feature importances (what drives detection) ===")
48     for feat, imp in sorted(model.feature_importances().items(),
49                             key=lambda kv: -kv[1]):
50         print(f"  {feat:16s}  {imp*100:5.1f}%")
51     return model
52
53 if __name__ == "__main__":
54     path = sys.argv[1] if len(sys.argv) > 1 else None
55     main(path)
```


ids/alerting.py

103 lines

```
1  """Phase 6 - real-time alerting.
2
3  When a high-confidence threat is detected the IDS can:
4  * pop a desktop notification (macOS),
5  * append to a human-readable alert log (alerts.log), and
6  * optionally send an email (if you set the env vars below).
7
8  Everything is OFF by default so the live demo isn't noisy. Turn it on:
9
10     IDS_ALERTS=1 python app.py           # desktop notifications + log
11     IDS_ALERT_MIN=0.9 IDS_ALERTS=1 ...   # only alert above this suspicion
12
13  Email (optional), all via environment variables:
14     IDS_SMTP_HOST, IDS_SMTP_PORT, IDS_SMTP_USER, IDS_SMTP_PASS, IDS_ALERT_TO
15 """
16 import os
17 import time
18 import subprocess
19 import smtplib
20 from email.mime.text import MIMEText
21
22 LOG_PATH = os.path.join(os.path.dirname(os.path.dirname(__file__)), "alerts.log")
23
24 # per-source cooldown so one attack burst doesn't spam 50 notifications
25 last_notified = {}
26 COOLDOWN = 0.0 # seconds
27
28
29 def enabled() -> bool:
30     return os.environ.get("IDS_ALERTS", "0") == "1"
31
32
33 def min_suspicion() -> float:
34     try:
35         return float(os.environ.get("IDS_ALERT_MIN", "0.85"))
36     except ValueError:
37         return 0.85
38
39
40 def _desktop(title, message):
41     """macOS notification via osascript; silently ignored elsewhere."""
42     try:
43         subprocess.run(
44             [
45                 "osascript", "-e",
46                 f'display notification "{message}" with title "{title}"',
47                 check=False, capture_output=True, timeout=3)
48     except Exception:
49         pass
50
51 def _email(subject, body):
52     host = os.environ.get("IDS_SMTP_HOST")
53     to = os.environ.get("IDS_ALERT_TO")
54     if not host or not to:
55         return
56     try:
57         msg = MIMEText(body)
58         msg["Subject"] = subject
59         msg["From"] = os.environ.get("IDS_SMTP_USER", "ids@lab.local")
60         msg["To"] = to
61         s = smtplib.SMTP(host, int(os.environ.get("IDS_SMTP_PORT", "25")), timeout=5)
62         user = os.environ.get("IDS_SMTP_USER"); pw = os.environ.get("IDS_SMTP_PASS")
63         if user and pw:
64             s.starttls(); s.login(user, pw)
65         s.sendmail(msg["From"], [to], msg.as_string()); s.quit()
66     except Exception:
67         pass
68
69
70 def _log(line):
71     try:
72         with open(LOG_PATH, "a") as fh:
73             fh.write(line + "\n")
74     except Exception:
75         pass
76
77
78 def maybe_alert(evt: dict):
79     """Called for every scored flow; acts only on strong anomalies when enabled."""
80     if not enabled() or not evt.get("anomaly"):
81         return
82     if evt.get("suspicion", 0) < min_suspicion():
83         return
84     src = evt.get("src", "?")
85     now = time.time()
86     if now - _last_notified.get(src, 0) < COOLDOWN:
87         _log_only(evt) # still log, just don't re-notify
88         return
89     _last_notified[src] = now
90     title = f"IDS ALERT · {evt.get('label', 'threat')}"
91     message = (f"{evt.get('src')} → {evt.get('dst')}: {evt.get('dst_port')} "
92              f"suspicion {evt.get('suspicion'):.2f}")
93     _desktop(title, message)
94     _email(title, message + f"\nbytes={evt.get('bytes')} packets={evt.get('packets')}")
95     _log_only(evt)
96
97
98 def _log_only(evt):
99     ts = time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(evt.get("ts", time.time())))
100     _log(f"[{ts}] {evt.get('label', 'threat')}: {evt.get('src')} -> "
101         f"{evt.get('dst')}: {evt.get('dst_port')}  suspicion={evt.get('suspicion'):.2f} "
102         f"source={evt.get('source', '?')}")
```

ids/capture.py

94 lines

```
1  """Live packet capture / pcap replay using Scapy.
2
3  This is the REAL capture path for your lab. It sniffs packets on an interface
4  (or reads a .pcap), aggregates them into flows, and POSTs each flow to the
5  running dashboard's /api/flow endpoint so it gets scored and displayed.
6
7  Scapy is imported lazily so the web app runs even where Scapy isn't installed
8  (e.g. this cloud sandbox). Install it in your lab with: pip install scapy
9
10 Examples
11 -----
12 # replay a capture file into the dashboard
13 python -m ids.capture --pcap lab_traffic.pcap --url http://127.0.0.1:5001
14
15 # live sniff (needs root/administrator + a real interface)
16 sudo python -m ids.capture --iface eth0 --url http://127.0.0.1:5001
17
18 SAFETY: only capture on interfaces/hosts you own, inside an isolated lab.
19 """
20 import argparse
21 import time
22
23 import requests
24
25 from .features import packets_to_flows
26
27
28 def _load_scapy():
29     try:
30         import scapy.all as scapy # noqa
31         return scapy
32     except Exception as e:
33         raise SystemExit(
34             "Scapy is required for live capture / pcap replay.\n"
35             "Install it in your lab: pip install scapy\n"
36             f"(import error: {e})"
37         )
38
39
40 def post_flows(flows, url):
41     endpoint = url.rstrip("/") + "/api/flow"
42     sent = 0
43     for f in flows:
44         try:
45             requests.post(endpoint, json=f, timeout=3)
46             sent += 1
47             time.sleep(0.05)
48         except requests.RequestException as e:
49             print(" ! could not reach dashboard:", e)
50             break
51     return sent
52
53
54 def from_pcap(path, url):
55     scapy = _load_scapy()
56     print(f"Reading {path} ...")
57     packets = scapy.rdpcap(path)
58     flows = packets_to_flows(packets)
59     print(f"Aggregated {len(flows)} flows; sending to {url} ...")
60     print("Sent", post_flows(flows, url), "flows.")
61
62
63 def from_iface(iface, url, window=5):
64     scapy = _load_scapy()
65     print(f"Sniffing on {iface} in {window}s windows (Ctrl-C to stop) ...")
66     try:
67         while True:
68             packets = scapy.sniff(iface=iface, timeout=window)
69             flows = packets_to_flows(packets)
70             print(f" window: {len(packets)} packets -> {len(flows)} flows")
71             post_flows(flows, url)
72     except KeyboardInterrupt:
73         print("\nStopped.")
74
75
76 def main():
77     ap = argparse.ArgumentParser(description="IDS capture -> dashboard bridge")
78     ap.add_argument("--pcap", help="read flows from a .pcap file")
79     ap.add_argument("--iface", help="live-sniff this interface (needs root)")
80     ap.add_argument("--url", default="http://127.0.0.1:5001",
81                     help="dashboard base URL")
82     ap.add_argument("--window", type=int, default=5, help="sniff window seconds")
83     args = ap.parse_args()
84     if args.pcap:
85         from_pcap(args.pcap, args.url)
86     elif args.iface:
87         from_iface(args.iface, args.url, args.window)
88     else:
89         ap.error("pass --pcap FILE or --iface NAME")
90
91
92 if __name__ == "__main__":
93     main()
```

114 lines

```
1  """Labeled datasets for the SUPERVISED model (Phase 4).
2
3  Two sources, same feature schema, so the Random Forest can train on whichever
4  you have:
5
6  1. synthetic_labeled() - normal + labelled attack flows from generator.py.
7     Fully self-contained; great for building/validating the pipeline and for
8     the demo. This is the default.
9
10 2. load_nslkdd(path) - parse the real, public NSL-KDD intrusion dataset
11   (KDDTrain+.txt / KDDTest+.txt) and map it onto our flow features. Download
12   it in your lab, drop the file in data/, and train on real research data.
13
14 3. (your lab) - once you capture labelled traffic in the lab
15   (Phase 3), save each flow as a row with a 'label' column and reuse
16   rows_to_xy(); the model doesn't care where the flows came from.
17 """
18 import csv
19 import random
20
21 from .features import FEATURES, flow_to_vector
22 from . import generator
23
24
25 # -----
26 # 1) Synthetic labelled data (default)
27 # -----
28 def synthetic_labeled(n_normal=4000, n_attack_each=700, seed=42):
29     """Return (X, y) where y is 'normal' or an attack-type string."""
30     rng = random.Random(seed)
31     X, y = [], []
32     for _ in range(n_normal):
33         f = generator.normal_flow(rng)
34         X.append(flow_to_vector(f)); y.append("normal")
35     for kind in generator.ATTACK_TYPES:
36         for _ in range(n_attack_each):
37             f = generator.attack_flow(kind, rng)
38             X.append(flow_to_vector(f)); y.append(kind)
39     # shuffle together
40     idx = list(range(len(X))); rng.shuffle(idx)
41     X = [X[i] for i in idx]; y = [y[i] for i in idx]
42     return X, y
43
44 # -----
45 # 2) Real public dataset: NSL-KDD
46 # -----
47 # -----
48 # NSL-KDD column order (41 features + label + difficulty). We only need a few
49 # columns and map them onto our 6 flow features.
50 NSLKDD_COLS = [
51     "duration", "protocol_type", "service", "flag", "src_bytes", "dst_bytes",
52     "land", "wrong_fragment", "urgent", "hot", "num_failed_logins", "logged_in",
53     "num_compromised", "root_shell", "su_attempted", "num_root",
54     "num_file_creations", "num_shells", "num_access_files", "num_outbound_cmds",
55     "is_host_login", "is_guest_login", "count", "srv_count", "serror_rate",
56     "srv_serror_rate", "rerror_rate", "srv_rerror_rate", "same_srv_rate",
57     "diff_srv_rate", "srv_diff_host_rate", "dst_host_count",
58     "dst_host_srv_count", "dst_host_same_srv_rate", "dst_host_diff_srv_rate",
59     "dst_host_same_src_port_rate", "dst_host_srv_diff_host_rate",
60     "dst_host_serror_rate", "dst_host_srv_serror_rate", "dst_host_rerror_rate",
61     "dst_host_srv_rerror_rate", "label", "difficulty",
62 ]
63 # NSL-KDD attack names -> our attack-family labels (coarse mapping).
64 _NSLKDD_FAMILY = {
65     "neptune": "SYN flood", "smurf": "SYN flood", "back": "SYN flood",
66     "ipsweep": "Port scan", "portsweep": "Port scan", "nmap": "Port scan", "satan": "Port scan",
67     "guess_passwd": "Brute force", "ftp_write": "Brute force", "imap": "Brute force",
68     "warezclient": "Data exfiltration", "warezmaster": "Data exfiltration", "multihop": "Data exfiltration",
69 }
70
71
72 def load_nslkdd(path, limit=None):
73     """Parse NSL-KDD (KDDTrain+.txt) into our (X, y). Unknown attacks map to a
74     generic 'attack' label; 'normal' stays 'normal'."""
75     X, y = [], []
76     with open(path) as fh:
77         reader = csv.reader(fh)
78         for i, row in enumerate(reader):
79             if limit and i >= limit:
80                 break
81             if len(row) < len(_NSLKDD_COLS) - 1:
82                 continue
83             rec = dict(zip(_NSLKDD_COLS, row))
84             duration = float(rec["duration"])
85             src = float(rec["src_bytes"]); dst = float(rec["dst_bytes"])
86             bytes_ = src + dst
87             packets = float(rec["count"]) or 1.0
88             flow = {
89                 "bytes": bytes_, "packets": packets, "duration": duration,
90                 "conn_rate": packets / duration if duration > 0 else packets,
91                 "dst_port": 0, # NSL-KDD encodes service, not port; left neutral
92             }
93             X.append(flow_to_vector(flow))
94             lbl = rec["label"].strip().lower()
95             y.append("normal" if lbl == "normal" else _NSLKDD_FAMILY.get(lbl, "attack"))
96     return X, y
97
98 # -----
99 # 3) Generic: rows with a label column -> (X, y) (for lab-captured data)
100 # -----
101 # -----
102 def rows_to_xy(rows):
103     """rows: iterable of dicts each having flow fields + 'label'."""
104     X, y = [], []
105     for r in rows:
106         X.append(flow_to_vector(r))
107         y.append(r.get("label", "normal"))
108     return X, y
109
110
111 def binary(y):
112     """Collapse multi-class labels to 1 = attack, 0 = normal."""
113     return [0 if lbl == "normal" else 1 for lbl in y]
```

60 lines

```
1 """Flow feature engineering for the IDS.
2
3 A *flow* is a summary of packets exchanged between two endpoints. We turn each
4 flow into a fixed numeric feature vector the ML model can score. Keeping this
5 in one place means the training script, the live scorer, and the Scapy capture
6 path all compute features identically.
7 """
8 from __future__ import annotations
9
10 # The model sees exactly these features, in this order.
11 FEATURES = ["bytes", "packets", "duration", "mean_pkt_size", "conn_rate", "dst_port"]
12
13
14 def flow_to_vector(flow: dict) -> list:
15     """Convert a flow dict into the ordered numeric vector the model expects."""
16     bytes_ = float(flow.get("bytes", 0))
17     packets = float(flow.get("packets", 1)) or 1.0
18     duration = float(flow.get("duration", 0.0))
19     mean_pkt = bytes_ / packets if packets else 0.0
20     conn_rate = float(flow.get("conn_rate", 0.0))
21     dst_port = float(flow.get("dst_port", 0))
22     return [bytes_, packets, duration, mean_pkt, conn_rate, dst_port]
23
24
25 def packets_to_flows(packets) -> list:
26     """Aggregate a list of Scapy packets into flow dicts keyed by
27     (src, dst, dst_port, proto). Used by the Scapy/pcap capture path.
28
29     `packets` is a Scapy PacketList. Imported lazily by the caller so this
30     module has no hard dependency on Scapy.
31     """
32     flows = {}
33     for pkt in packets:
34         try:
35             if not pkt.haslayer("IP"):
36                 continue
37             ip = pkt["IP"]
38             proto = "TCP" if pkt.haslayer("TCP") else "UDP" if pkt.haslayer("UDP") else "OTHER"
39             dport = 0
40             if pkt.haslayer("TCP"):
41                 dport = int(pkt["TCP"].dport)
42             elif pkt.haslayer("UDP"):
43                 dport = int(pkt["UDP"].dport)
44             key = (ip.src, ip.dst, dport, proto)
45             f = flows.setdefault(key, {
46                 "src": ip.src, "dst": ip.dst, "dst_port": dport, "proto": proto,
47                 "bytes": 0, "packets": 0, "t_first": float(pkt.time), "t_last": float(pkt.time),
48             })
49             f["bytes"] += int(len(pkt))
50             f["packets"] += 1
51             f["t_last"] = float(pkt.time)
52         except Exception:
53             continue
54     out = []
55     for f in flows.values():
56         f["duration"] = max(0.0, f["t_last"] - f["t_first"])
57         f["conn_rate"] = f["packets"] / f["duration"] if f["duration"] > 0 else float(f["packets"])
58         out.append(f)
59     return out
```

```
1  """Synthetic traffic generator.
2
3  Produces realistic 'normal' flows plus labelled attack flows so you can (a)
4  train the model on normal traffic and (b) drive the live dashboard demo without
5  needing a real network. In a real lab you'd replace this with Scapy capture
6  (see ids/capture.py) - the feature schema is identical, so the model doesn't
7  care where flows come from.
8  """
9  import random
10
11  NORMAL_PORTS = [80, 443, 443, 443, 53, 22, 3389, 445, 8080]
12  INTERNAL_IPS = ["10.0.2.4", "10.0.2.7", "10.0.2.9", "10.0.2.15", "192.168.56.101"]
13  DESTS = ["10.0.2.2", "10.0.2.3", "10.0.2.5", "10.0.2.6", "8.8.8.8", "140.82.113.20"]
14
15  ATTACK_TYPES = ["Port scan", "SYN flood", "Data exfiltration", "Brute force", "Beaconing"]
16
17  def normal_flow(rng=random) -> dict:
18      """A believable benign flow: moderate bytes, low connection rate."""
19      port = rng.choice(NORMAL_PORTS)
20      packets = rng.randint(4, 120)
21      bytes_ = int(packets * rng.uniform(120, 900))
22      duration = rng.uniform(0.4, 25.0)
23      return {
24          "src": rng.choice(INTERNAL_IPS), "dst": rng.choice(DESTS),
25          "dst_port": port, "proto": "UDP" if port == 53 else "TCP",
26          "bytes": bytes_, "packets": packets, "duration": round(duration, 2),
27          "conn_rate": round(packets / duration, 2), "label": "normal",
28      }
29
30
31
32  def attack_flow(kind=None, rng=random) -> dict:
33      kind = kind or rng.choice(ATTACK_TYPES)
34      src = rng.choice(INTERNAL_IPS)
35      dst = rng.choice(DESTS)
36      if kind == "Port scan":
37          packets = rng.randint(1, 3)
38          f = {"dst_port": rng.randint(1, 65535), "bytes": packets * rng.randint(40, 74),
39              "packets": packets, "duration": round(rng.uniform(0.01, 0.2), 3)}
40      elif kind == "SYN flood":
41          packets = rng.randint(400, 3000)
42          dur = rng.uniform(0.5, 2.0)
43          f = {"dst_port": rng.choice([80, 443]), "bytes": packets * 60,
44              "packets": packets, "duration": round(dur, 2)}
45      elif kind == "Data exfiltration":
46          packets = rng.randint(200, 900)
47          f = {"dst_port": rng.choice([443, 22, 53]), "bytes": rng.randint(4_000_000, 40_000_000),
48              "packets": packets, "duration": round(rng.uniform(3, 20), 2)}
49      elif kind == "Brute force":
50          packets = rng.randint(120, 600)
51          dur = rng.uniform(2, 8)
52          f = {"dst_port": rng.choice([22, 3389, 445]), "bytes": packets * rng.randint(120, 300),
53              "packets": packets, "duration": round(dur, 2)}
54      else: # Beaconing (regular tiny callouts to a rare port)
55          packets = rng.randint(2, 6)
56          f = {"dst_port": rng.choice([4444, 8081, 6667]), "bytes": packets * rng.randint(80, 200),
57              "packets": packets, "duration": round(rng.uniform(0.05, 0.3), 3)}
58      dur = f["duration"] or 0.01
59      f.update({"src": src, "dst": dst, "proto": "TCP",
60              "conn_rate": round(f["packets"] / dur, 2), "label": kind})
61      return f
62
63
64  def stream(n=None, attack_prob=0.10, rng=random):
65      """Yield flows forever (or n of them), mixing normal + attacks."""
66      made = 0
67      while n is None or made < n:
68          yield attack_flow(rng=rng) if rng.random() < attack_prob else normal_flow(rng=rng)
69          made += 1
```

```
1  """The anomaly-detection model.
2
3  Wraps a scikit-learn IsolationForest trained on NORMAL traffic only
4  (unsupervised). At inference it returns a 0..1 suspicion score and a boolean
5  'anomaly' verdict. The model bundle (estimator + score normalisation) is saved
6  to model.pkl by train.py and loaded here.
7  """
8  import os
9  import pickle
10 import numpy as np
11 from sklearn.ensemble import IsolationForest
12
13 from .features import FEATURES, flow_to_vector
14
15 MODEL_PATH = os.path.join(os.path.dirname(os.path.dirname(__file__)), "model.pkl")
16
17 class IDSMModel:
18     def __init__(self, estimator=None, lo=0.0, hi=1.0, threshold=0.5):
19         self.estimator = estimator
20         self.lo = lo          # min raw anomaly score seen in training
21         self.hi = hi          # max raw anomaly score seen in training
22         self.threshold = threshold
23
24     # ---- training -----
25     @classmethod
26     def train(cls, normal_flows, contamination=0.06, random_state=42):
27         X = np.array([flow_to_vector(f) for f in normal_flows], dtype=float)
28         # log-scale the heavy-tailed features so bytes/packets don't dominate
29         Xs = _prescale(X)
30         est = IsolationForest(n_estimators=200, contamination=contamination,
31                               random_state=random_state)
32         est.fit(Xs)
33         raw = -est.score_samples(Xs)          # higher = more anomalous
34         lo, hi = float(raw.min()), float(raw.max())
35         # threshold: the (1-contamination) quantile of normal suspicion
36         thr = float(np.quantile(raw - lo) / (hi - lo + 1e-9), 1 - contamination))
37         return cls(est, lo, hi, thr)
38
39     # ---- inference -----
40     def score(self, flow: dict):
41         x = _prescale(np.array([flow_to_vector(flow)], dtype=float))
42         raw = float(-self.estimator.score_samples(x)[0])
43         susp = (raw - self.lo) / (self.hi - self.lo + 1e-9)
44         susp = max(0.0, min(1.0, susp))
45         return susp, bool(susp >= self.threshold)
46
47     # ---- persistence -----
48     def save(self, path=MODEL_PATH):
49         with open(path, "wb") as fh:
50             pickle.dump({"estimator": self.estimator, "lo": self.lo,
51                           "hi": self.hi, "threshold": self.threshold,
52                           "features": FEATURES}, fh)
53
54     @classmethod
55     def load(cls, path=MODEL_PATH):
56         with open(path, "rb") as fh:
57             d = pickle.load(fh)
58             return cls(d["estimator"], d["lo"], d["hi"], d["threshold"])
59
60 def _prescale(X):
61     """loglp the size-like columns (bytes, packets, mean_pkt_size, conn_rate,
62     dst_port) so their huge dynamic range doesn't swamp the forest."""
63     X = X.copy()
64     for col in (0, 1, 3, 4, 5): # bytes, packets, mean_pkt_size, conn_rate, dst_port
65         X[:, col] = np.loglp(np.clip(X[:, col], 0, None))
66     return X
```


ids/supervised.py

70 lines

```
1  """Supervised classifier for the IDS (Phase 4): a Random Forest.
2
3  Unlike the Isolation Forest (which learns only 'normal' and flags outliers),
4  the Random Forest learns from LABELLED normal + attack flows, so it can also
5  say *which kind* of attack it thinks a flow is, and it exposes feature
6  importances that explain its decisions.
7  """
8  import os
9  import pickle
10
11  import numpy as np
12  from sklearn.ensemble import RandomForestClassifier
13
14  from .features import FEATURES, flow_to_vector
15  from .model import _prescale # reuse the same log-scaling as the anomaly model
16
17  MODEL_PATH = os.path.join(os.path.dirname(os.path.dirname(__file__)), "rf_model.pkl")
18
19
20  class RFModel:
21      def __init__(self, clf=None, classes=None):
22          self.clf = clf
23          self.classes = list(classes) if classes is not None else None
24
25      @classmethod
26      def train(cls, X, y, n_estimators=300, random_state=42):
27          Xs = _prescale(np.array(X, dtype=float))
28          clf = RandomForestClassifier(
29              n_estimators=n_estimators, random_state=random_state,
30              class_weight="balanced_subsample", n_jobs=-1)
31          clf.fit(Xs, y)
32          return cls(clf, clf.classes_)
33
34      def predict(self, X):
35          Xs = _prescale(np.array(X, dtype=float))
36          return self.clf.predict(Xs)
37
38      def predict_proba(self, X):
39          Xs = _prescale(np.array(X, dtype=float))
40          return self.clf.predict_proba(Xs)
41
42      def attack_probability(self, X):
43          """Return P(flow is any attack) = 1 - P(normal) for each row."""
44          proba = self.predict_proba(X)
45          classes = list(self.clf.classes_)
46          if "normal" in classes:
47              j = classes.index("normal")
48              return 1.0 - proba[:, j]
49          return proba.max(axis=1)
50
51      def feature_importances(self):
52          return dict(zip(FEATURES, self.clf.feature_importances_))
53
54      # ---- convenience for a single flow dict (used by the live app) ----
55      def score_flow(self, flow: dict):
56          x = [flow_to_vector(flow)]
57          p = float(self.attack_probability(x)[0])
58          label = str(self.predict(x)[0])
59          return p, label
60
61      def save(self, path=MODEL_PATH):
62          with open(path, "wb") as fh:
63              pickle.dump({"clf": self.clf, "classes": self.classes}, fh)
64
65      @classmethod
66      def load(cls, path=MODEL_PATH):
67          with open(path, "rb") as fh:
68              d = pickle.load(fh)
69          return cls(d["clf"], d["classes"])
```

templates/dashboard.html

161 lines

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>IDS Live Dashboard · CyberLab Project 1</title>
7 <style>
8 root{--bg:#0b1220;--panel:#131c31;--panel-2:#1b2743;--ink:#eaf0ff;--muted:#9ab6d6;
9 --line:#2a3a5f;--accent:#5eb0ff;--accent-ink:#04101f;--ok:#3bd39f;--warn:#ffcfc5;--crit:#ff6b7d;--focus:#ffd23f}
10 @media (prefers-reduced-motion: reduce){*{animation:none!important;transition:none!important}}
11 *(box-sizing:border-box)
12 body{margin:0;background:linear-gradient(180deg,#0a1120,#0b1220 40%);color:var(--ink);
13 font:16px/1.6 system-ui,-apple-system,Segoe UI,Roboto,Helvetica,Arial,sans-serif}
14 a{color:var(--accent)}
15 button:focus-visible,select:focus-visible,[tabindex]:focus-visible{outline:3px solid var(--focus);outline-offset:2px;border-radius:6px}
16 .skip{position:absolute;left:-999px;top:0;background:var(--focus);color:#111;padding:.6rem 1rem;z-index:9;font-weight:700}
17 .skip:focus{left:0}
18 header{padding:1.4rem 1.2rem .4rem;max-width:1100px;margin:0 auto}
19 h1{font-size:1.6rem;margin:.2rem 0}
20 .muted{color:var(--muted)} .eyebrow{color:var(--accent);font-weight:700;letter-spacing:.1em;text-transform:uppercase;font-size:.72rem}
21 main{max-width:1100px;margin:0 auto;padding:1rem 1.2rem 3rem}
22 .controls{display:flex;gap:.6rem;flex-wrap:wrap;align-items:center;margin:.6rem 0}
23 button.btn{background:var(--accent);color:var(--accent-ink);border:0;font-weight:700;padding:.55rem 1rem;border-radius:10px;cursor:pointer}
24 button.btn_ghost{background:transparent;color:var(--ink);border:1px solid var(--line)}
25 select{background:#0426;boder:1px solid var(--line);color:var(--ink);border-radius:8px;padding:.4rem}
26 .grid{display:grid;gap:1rem}
27 @media(min-width:760px){cols:4{grid-template-columns:repeat(4,1fr)}.cols:2{grid-template-columns:1fr 1fr}}
28 .stat{background:var(--panel-2);border:1px solid var(--line);border-radius:14px;padding:1rem}
29 .stat .n{font-size:2rem;font-weight:800;font-variant-numeric:tabular-nums}
30 .stat .l{color:var(--muted);font-size:.85rem}
31 .panel{background:var(--panel);border:1px solid var(--line);border-radius:14px;padding:1.1rem}
32 .bars{display:flex;align-items:flex-end;gap:5px;height:120px}
33 .bars .bar{flex:1;background:linear-gradient(180deg,var(--accent),#2b5c96);border-radius:5px 5px 0 0;min-height:2px}
34 .bars .bar_hot{background:linear-gradient(180deg,var(--crit),#a2130)}
35 .feed{max-height:320px;overflow:auto;border:1px solid var(--line);border-radius:10px;background:#0a1428}
36 table{width:100%;border-collapse:collapse;font-size:.86rem}
37 th,td{text-align:left;padding:.45rem .6rem;border-bottom:1px solid #16223f;font-variant-numeric:tabular-nums}
38 th{position:sticky;top:0;background:#0e1a34;color:var(--muted)}
39 tr.crit td:first-child{box-shadow:inset 3px 0 0 var(--crit)}
40 tr.warn td:first-child{box-shadow:inset 3px 0 0 var(--warn)}
41 .pill{display:inline-block;font-size:.72rem;font-weight:700;padding:.15rem .5rem;border-radius:999px;border:1px solid}
42 .p-crit{color:#ff6b7d;border-color:var(--crit);background:rgba(255,107,125,.16)}
43 .p-warn{color:#ffe6a6;border-color:var(--warn);background:rgba(255,207,92,.16)}
44 .p-ok{color:#a6f0d6;border-color:var(--ok);background:rgba(56,211,159,.16)}
45 .dot{display:inline-block;width:.6rem;height:.6rem;border-radius:50%}
46 .visually-hidden{position:absolute;width:1px;height:1px;overflow:hidden;clip:rect(0 0 0 0)}
47 </style>
48 </head>
49 <body>
50 <a class="skip" href="#main">Skip to main content</a>
51 <header>
52 <div style="display:flex;align-items:center;gap:.6rem">
53 
54 <p class="eyebrow" style="margin:0">Katy CyberWise · Project 1 · Live</p>
55 </div>
56 <h1>Network Intrusion Detection – Live Dashboard</h1>
57 <p class="muted">Flows are scored by a trained Isolation Forest as they arrive over Server-Sent Events.
58 <span id="conn" class="pill p-warn">connecting</span></p>
59 </header>
60 <main id="main">
61 <div class="controls" role="group" aria-label="Controls">
62 <button class="btn" id="pauseBtn" aria-pressed="false">Pause</button>
63 <button class="btn_ghost" id="attackBtn">Inject attack burst</button>
64 <label class="muted" style="margin-left:auto">Speed
65 <select id="speed" aria-label="Stream speed">
66 <option value="1.2">Slow</option><option value="0.7" selected>Normal</option><option value="0.3">Fast</option>
67 </select>
68 </label>
69 </div>
70
71 <div class="grid cols-4" style="margin-bottom:1rem">
72 <div class="stat"><div class="n" id="s-flows">0</div><div class="l">Flows analysed</div></div>
73 <div class="stat"><div class="n" id="s-alerts" style="color:var(--crit)">0</div><div class="l">Threat alerts</div></div>
74 <div class="stat"><div class="n" id="s-rate">0</div><span style="font-size:1rem">/min</span></div><div class="l">Alert rate</div></div>
75 <div class="stat"><div class="n" id="s-status">Live</div><div class="l">Monitor status</div></div>
76 </div>
77
78 <div class="grid cols-2">
79 <div class="panel">
80 <h2 style="margin-top:0;font-size:1.1rem">Traffic volume (recent windows)</h2>
81 <div class="bars" id="img" role="img" aria-label="Traffic volume per window; red bars contain alerts"></div>
82 <p class="muted" style="font-size:.85rem"><span class="dot" style="background:var(--accent)"></span> normal &nbsp; <span class="dot" style="background:var(--crit)"></span> window with
alert</p>
83 </div>
84 <div class="panel">
85 <h2 style="margin-top:0;font-size:1.1rem">Alerts by types</h2>
86 <div id="breakdown" aria-live="polite"></div>
87 </div>
88 </div>
89
90 <div class="panel" style="margin-top:1rem">
91 <h2 style="margin-top:0;font-size:1.1rem">Live flow feed</h2>
92 <p class="visually-hidden" aria-live="polite" id="live">Waiting for data.</p>
93 <div class="feed" tabindex="0" role="region" aria-label="Live flow feed">
94 <table>
95 <caption class="visually-hidden">Recent flows with suspicion score and verdict</caption>
96 <thead><tr><th>Time</th><th>Source → Dest</th><th>Proto</th><th>Port</th><th>Bytes</th><th>Score</th><th>Verdict</th></tr></thead>
97 <tbody id="feed"><tr><td colspan="7" class="muted" style="text-align:center;padding:1rem">Connecting to stream.</td></tr></tbody>
98 </table>
99 </div>
100 <p class="muted" style="font-size:.85rem;margin-bottom:0">Source of each flow: <code>demo</code> = built-in generator, <code>capture</code> = real traffic from the Scapy bridge.</p>
101 </div>
102 </main>
103
104 <script>
105 function(){
106 "use strict";
107 var reduce = matchMedia("(prefers-reduced-motion: reduce)").matches;
108 var feed=document.getElementById("feed"),chart=document.getElementById("chart"),breakdown=document.getElementById("breakdown"),live=document.getElementById("live");
109 var sFlows=document.getElementById("s-flows"),sAlerts=document.getElementById("s-alerts"),sRate=document.getElementById("s-rate"),sStatus=document.getElementById("s-
status"),conn=document.getElementById("conn");
110 var types={},windowVals=[],winDownAlert=[],curWin=0,winCount=0,winHot=false,alertTimes=[],first=true;
111
112 function fmtBytes(n){return n>=1e6?(n/1e6).toFixed(1)+" MB":n>=1e3?(n/1e3).toFixed(1)+" KB":n+" B";}
113 function hhmmss(ts){var d=new Date(ts*1000);return [d.getHours(),d.getMinutes(),d.getSeconds()].map(function(x){return String(x).padStart(2,"0")}).join(":");}
114
115 function renderChart(){chart.innerHTML="";var mx=Math.max.apply(null,windowVals.concat([1]));
116 windowVals.forEach(function(v,i){var b=document.createElement("div");b.className="bar"+(windowAlert[i]?" hot":"");b.style.height=Math.max(2,v/mx*118)+"px";chart.appendChild(b)});}
117 function renderBreakdown(){var keys=Object.keys(types);if(!keys.length){breakdown.innerHTML="<p class='muted'>No alerts yet.</p>";return;}
118 var mx=Math.max.apply(null,keys.map(function(k){return types[k]}));
119 breakdown.innerHTML=keys.map(function(k){var p=types[k]/mx*100;
120 return <div style="background:35rem 0"><div style="display:flex;justify-content:space-between;font-size:.85rem"><span>+</span><span class="muted">+types[k]+</span></div><div
style="height:10px;background:#0a1428;border:1px solid var(--line);border-radius:6px;overflow:hidden"><div style="height:100%;width:1+p+%;background:linear-gradient(90deg,var(--crit),#a2130)"></div>
</div></div>});}.join("");}
121
122 function onFlow(e){
123 if(first){feed.innerHTML="";first=false;}
124 var f=JSON.parse(e.data);
125 // stats
126 sFlows.textContent=(f.total_flows||0).toLocaleString();
127 sAlerts.textContent=(f.total_alerts||0).toLocaleString();
128 // window chart
129 curWin+=f.bytes; if(f.anomaly)winHot=true; winCount++;
130 if(winCount==4){windowVals.push(curWin);windowAlert.push(winHot);if(windowVals.length>24){windowVals.shift();windowAlert.shift();}curWin=0;winCount=0;winHot=false;renderChart();}
131 // alert type + rate
132 if(f.anomaly){types[f.label]=(types[f.label]||0)+1;alertTimes.push(Date.now());renderBreakdown();}
133 var cut=Date.now()-60000;alertTimes=alertTimes.filter(function(t){return t>cut;});}
```

```
134 sRate.innerHTML=alertTimes.length+'<span style="font-size:1rem">/min</span>';
135 // feed row
136 var tr=document.createElement("tr");
137 if(f.anomaly)tr.className=f.suspicion>0.75?"crit":"warn";
138 var verdict=f.anomaly?'<span class="pill '+f.suspicion>0.75?"p-crit":"p-warn">'+f.label+'</span>':<span class="pill p-ok">normal</span>';
139 tr.innerHTML="<td>"+hhmmss(f.ts)+"</td><td>"+f.src+" → "+f.dst+"</td><td>"+f.proto+"</td><td>"+f.dst_port+"</td><td>"+fmtBytes(f.bytes)+"</td><td>"+f.suspicion.toFixed(2)+"</td><td>"+verdict+"
</td>";
140 feed.insertBefore(tr,feed.firstChild);
141 while(feed.children.length>60)feed.removeChild(feed.lastChild);
142 live.textContent="Latest: "+(f.anomaly?f.label+" from "+f.src:"normal")+", suspicion "+f.suspicion.toFixed(2)+".";
143 }
144
145 var es=new EventSource("/stream");
146 es.onopen=function(){conn.textContent="live";conn.className="pill p-ok";};
147 es.onerror=function(){conn.textContent="reconnecting...";conn.className="pill p-warn";};
148 es.onmessage=onFlow;
149
150 function post(action,extra){var b=new URLSearchParams();b.set("action",action);if(extra)Object.keys(extra).forEach(function(k){b.set(k,extra[k]);});
151 fetch("/control",{method:"POST",headers:{"Content-Type":"application/x-www-form-urlencoded"},body:b.toString()});}
152 var paused=false,pb=document.getElementById("pauseBtn");
153 pb.addEventListener("click",function(){paused=!paused;post(paused?"pause":"resume");pb.textContent=paused?"Resume":"Pause";pb.setAttribute("aria-
pressed",String(paused));sStatus.textContent=paused?"Paused":"Live";sStatus.style.color=paused?"var(--warn)":"var(--ok)";});
154 document.getElementById("attackBtn").addEventListener("click",function(){post("attack");live.textContent="Attack burst injected.";});
155 document.getElementById("speed").addEventListener("change",function(){post("speed",{value:this.value});});
156 renderBreakdown();
157 })();
158 </script>
159 </body>
160 </html>
```

27 lines

```
1 IDS evaluation summary
2 =====
3
4 Test flows: 1800 (normal + 5 attack types)
5
6 Random Forest ROC-AUC=1.000 AvgPrecision=1.000
7 Isolation Forest ROC-AUC=0.977 AvgPrecision=0.975
8
9 Detection rate by attack type (%):
10 Port scan RandomForest=100.0 IsolationForest=100.0
11 SYN flood RandomForest=100.0 IsolationForest=100.0
12 Data exfiltration RandomForest=100.0 IsolationForest= 63.3
13 Brute force RandomForest=100.0 IsolationForest= 70.6
14 Beaconing RandomForest=100.0 IsolationForest=100.0
15
16 Feature importances (Random Forest):
17 mean_pkt_size 27.7%
18 bytes 21.3%
19 duration 18.7%
20 packets 17.8%
21 conn_rate 9.4%
22 dst_port 5.1%
23
24 Note: figures use labelled synthetic traffic so the pipeline is
25 fully reproducible. Numbers on real NSL-KDD or lab-captured traffic
26 will be lower/noisier – that is expected and more realistic.
```

requirements.txt

8 lines

```
1 Flask>=3.0
2 scikit-learn>=1.3
3 numpy>=1.24
4 requests>=2.28
5 matplotlib>=3.6
6 # Only needed for live packet capture / pcap replay in your lab:
7 scapy>=2.5
```